10. numpy

Julia Gallucci HB.Sc, M.Sc, Ph.D.

Campbell Family Mental Health Research Institute, CAMH

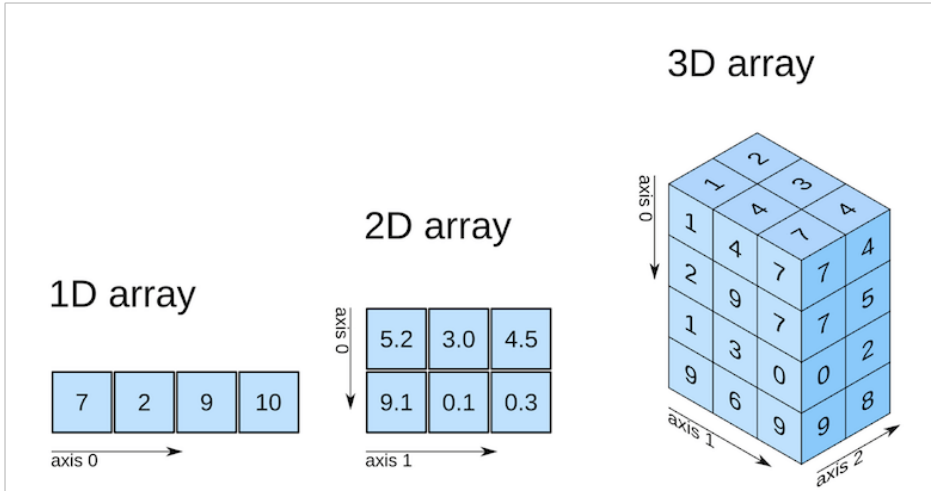
Institute of Medical Science | University of Toronto

Learning objectives

- Be able to understand understand the NumPy array
- Create and manipulate arrays in multiple ways
- Perform computations on arrays and avoid unnecessary loops by leveraging array operations

What is a NumPy array?

- `numpy` is a Python package used for **scientific and numerical computing**.
- It introduces a powerful data structure called an **array**, which is designed to store and work with numbers efficiently.
- A **NumPy array** is:
  - A grid of values
  - All values are the same data type
  - Can have one or more dimensions (1D, 2D, 3D, ...)
  - Think of it as a more efficient list for numbers.



Loading NumPy

- We usually import NumPy with an alias:

```
import numpy as np
```

- This lets us write np instead of numpy, which is shorter and easier to read.

Arrays vs lists

NumPy Arrays	Python lists
All elements **must be the same data type**	Elements can be different types
Fixed size	Can grow or shrink
Can be **multidimensional**	Usually, 1D
Very fast for math operations	Slower for math
Uses **less memory**	Uses more memory

Arrays vs lists

NumPy Arrays	Python lists
All elements **must be the same data type**	Elements can be different types
Fixed size	Can grow or shrink
Can be **multidimensional**	Usually, 1D
Very fast for math operations	Slower for math
Uses **less memory**	Uses more memory

```
# Python list
numbers_list = [1, 2, 3, 4, 5]

# NumPy array
numbers_array = np.array([1, 2, 3, 4, 5])
```

1	2	3	4	5

1	2	3	4	5

Creating NumPy arrays

Most arrays we'll work with will be one-dimensional *vectors* and two-dimensional *matrices*.

```
# 1 row x 5 columns
vector = np.array([1, 2, 3, 4, 5])
# vector.ndim = 1, vector.shape = (5,1), vector.size = 5
```

1	**2**	**3**	**4**	**5**

```
# 2 rows x 3 columns
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
matrix
# matrix.ndim = 2, matrix.shape = (2,3), matrix.size = 6
```

1	**2**	**3**
4	**5**	**6**

- **ndim** attribute indicating the number of its axes
- **size** attribute indicating the number of values it has
- **shape** indicating its size in each dimension.

Placeholder Arrays in NumPy

- Sometimes we know the size of the array but not the values yet
- NumPy arrays have a fixed size, so we often initialize first
- We can use 0s as a placeholder

```
# NumPy array of zeros
np.zeros((2,3))
```

0	0	0
0	0	0

- Or 1s

```
# NumPy array of ones
np.ones((2,3))
```

1	1	1
1	1	1

Specific Number Arrays in NumPy

- We can also create a sequence of numbers using `arange()`

```
# NumPy array from 1 until 10 in steps of 2
np.arange(1,10,2)
```

1	3	5	7	9

Tip: arrange(start, stop, step)

- Or generating random numbers through functions like `random.randint()`

```
# NumPy array of 5 random integers between 1 and 10
np.random.randint(1,10,5)
```

6	9	6	1	1

Tip: random functions → generate unpredictable numbers. We use numpy's random seed function to create a reproducible random example

Repeat Arrays in NumPy

- We can repeat values and arrays to create bigger ones using `repeat()`

```
# NumPy array of 10 repeated 5 times
np.repeat(10,5)
```

10	10	10	10	10

Tip: np.repeat(number to repeat, iterations)

- tile repeats the whole input array for a specified number

```
# NumPy array repeated x times
np.tile(vector, 2)
```

1	2	3	4	5	1	2	3	4	5

Tip: np.tile(array, reps)

Simulating data in NumPy

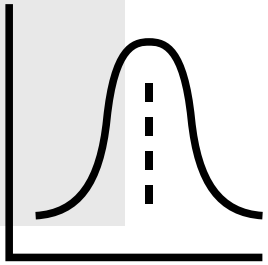
- Simulating Data with NumPy
  - Useful for testing analyses or models
  - NumPy supports drawing random samples from various distributions

- Example, creating a random number generator

```
# Create a Generator
rng = np.random.default_rng()
```

- From this, you can specify the distribution you want to sample from

```
# 10 data points from normal distributed data with a
mean of 0, sd 1
rng.normal(0,1,10)
# can also use uniform distribution, poisson etc
```



Reshaping arrays

- We can change the shape of an array in various ways, while leaving elements the same.

- We can **transpose an array**

```
# 2 rows x 3 columns
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
# 3 rows x 2 column
matrix.T
```

1	2	3
4	5	6

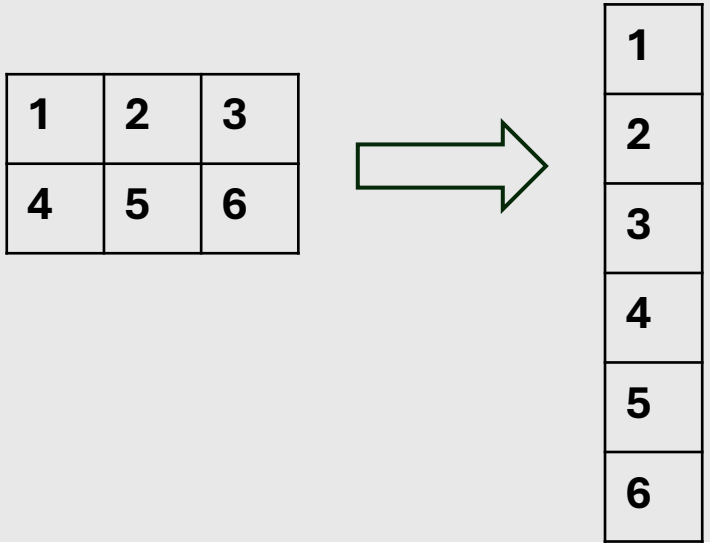
1	4
2	5
3	6

Reshaping arrays

- We can change the shape of an array in various ways, while leaving elements the same.

- We can **change the dimensions**

```
# 2 rows x 3 columns
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
# 6 rows x 1 column
matrix.reshape(6,1)
```

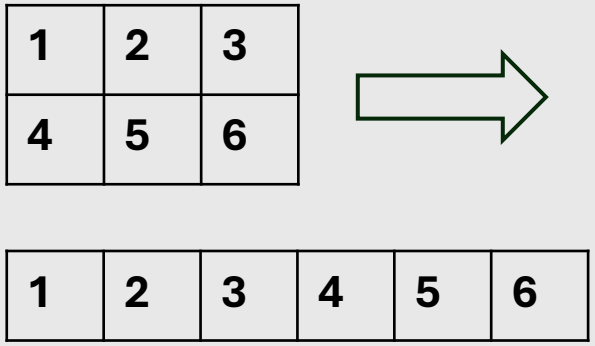


Reshaping arrays

- We can change the shape of an array in various ways, while leaving elements the same.

- We can **collapse an array** to one dimension

```
# 2 rows x 3 columns
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
# 1 row x 6 columns
matrix.flatten()
```



Reshaping arrays

- We can change the shape of an array in various ways, while leaving elements the same.

- If two arrays share the same size along an axis, we can **stack them either horizontally or vertically**

```
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6]])

matrix2 = np.array([[7, 8, 9],
                    [10, 11, 12]])

np.vstack((matrix1,matrix2)) #vertical stack
```

1	2	3
4	5	6

7	8	9
10	11	12

1	2	3
4	5	6
7	8	9
10	11	12

Reshaping arrays

- We can change the shape of an array in various ways, while leaving elements the same.

- If two arrays share the same size along an axis, we can **stack them either horizontally or vertically**

```
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6]])

matrix2 = np.array([[7, 8, 9],
                    [10, 11, 12]])

np.hstack((matrix1,matrix2)) #horizontal stack
```

1	2	3
4	5	6

7	8	9
10	11	12

1	2	3	7	8	9
4	5	6	10	11	12

Why NumPy is useful

- With NumPy, you can apply operations to all values at once

```
# NumPy array multiplied by 2
numbers_array = np.array([1, 2, 3, 4, 5])

numbers_array * 2
array([ 2,  4,  6,  8, 10])
```

1	**2**	**3**	**4**	**5**	x 2
2	**4**	**6**	**8**	**10**	

- Doing this with a list requires a loop.

Why NumPy is useful

- We can also perform operations when arrays are the same length

```
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6]])

matrix2 = np.array([[7, 8, 9],
                    [10, 11, 12]])

matrix2 - matrix1
```

1	2	3
4	5	6

7	8	9
10	11	12

6	6	6
6	6	6

Why NumPy is useful

- We can also summarize values in an array

```
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6]])

matrix1.sum()  #1 + 2 + 3 + 4 + 5 + 6 = 21
matrix1.max()  #6
matrix1.mean() # 1 + 2 + 3 + 4 + 5 + 6 / 6 = 3.5
```

1	**2**	**3**
4	**5**	**6**

Operations in multiple dimensions

- We can be more specific with how we calculate for multidimensional arrays
- We use **axis** to specify the direction (0 → column-wise, 1 row-wise operations)

```
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6]])

matrix1.mean(axis=0)

matrix1.mean(axis=1)
```

1	**2**	**3**
4	**5**	**6**

2.5 3.5 4.5

1	**2**	**3**	2
4	**5**	**6**	5

Operations in multiple dimensions

- We can index and slice arrays similar to a list
- Indexing goes [row, column]

```
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6]])

matrix1[0,:] #first row

matrix[:,1] #second column

matrix[1,1] #second row, second column
```

1	2	3
4	5	6

1	2	3
4	5	6

1	2	3
4	5	6

Operations in multiple dimensions

- We can mutate or update elements in an array
- We can also make a copy to ensure the original does not get modified

```
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6]])

matrix3 = matrix1.copy()
matrix3[1,1] = 50 #replace 5 with 50, original not changed
```

1	2	3
4	50	6

Operations in multiple dimensions

- E.g. Evaluate whether each element is divisible by 3

```
matrix3 % 3 == 0
array([[False, False, True],
[False, False, True]])
```

1	2	3
4	50	6

F	F	T
F	F	T

- To filter, use a Boolean expression as a mask, pass it into square brackets after the array to mask.

```
matrix3[matrix3 % 3 == 0]
array([3,6])
```

Operations in multiple dimensions

- We can even use masks **to generate new arrays with conditionals**.
- `np.where()` takes as its arguments a Boolean expression, an expression to evaluate if `True`, and an expression to evaluate elsewise.
- This is analogous to creating a new array based on an old one with a `for` loop and `if`/`else` statements, but much faster.

```
np.where(matrix3 % 3 == 0, # condition
matrix3, # return the element if True
0) # return 0 if False

array([[ 0, 0, 3], [ 0, 0, 6]])
```

1	2	3
4	50	6

0	0	3
0	0	6

Quiz: Arrays

1. **What is the shape of this array?**

```
np.array([[1, 2, 3], [4, 5, 6]])
```

A. (3,2)
B. (2,3)
C. (6,)
D. (3,3)

Quiz: Arrays

2. What does the following produce?

```
arr = np.array([10,20,30,40])
arr[2]
```

A. 10

B. 20

C. 30

D. 40

Quiz: Arrays

3. What does the following produce?

```
arr = np.array([1,2,3])
arr* 2
```

A. 2,4,6

B. 1,2,3,2,4,6

C. 3,4,5

D. Error

Quiz: Arrays

4. What does the following produce?

```
arr = np.zeros((2,3))
arr
```

A. 2x3 array of zeros

B. 3x2 array of zeros

C. Array [0,0,0]

D. Error

Quiz: Arrays

5. What does the following produce?

```
arr = np.array([1,2,3,4,5,6])
arr.reshape((2,3))
```

A.

1	2	3
4	5	6

B.

1	2	3	4	5	6

C.

1	2
3	4
5	6

D.

1	3	5
2	4	6

Exercise # 1

- Try this on your own:
  - Create a 1D array of numbers from 1 to 20.
  - Compute the sum, mean, and standard deviation of the array (*Note*, these are all numpy methods!)

Exercise # 2

- Try this on your own:
  - From the following 2D array

```
arr = np.array([[2, 7, 12],
                [4, 15, 3],
                [8, 1, 20]])
```

  - Select all elements greater than 10
  - Replace all elements less than 5 with 0

Exercise # 3

- Try this on your own:
  - From the following 2D array

```
arr = np.array([[2, 7, 12],
                [4, 15, 3],
                [8, 1, 20]])
```

  - Print the first column
  - Print the last row
  - Print the element in the 2nd row, 3rd column

Exercise # 4

- Try this on your own:
  - From the following 2D array

```
arr = np.array([[2, 7, 12],
                [4, 15, 3],
                [8, 1, 20]])
```

  - Compute the sum of each column
  - Compute the mean of each row

Remember:
axis 0 = col,
axis 1 = row

Python For Data Science *Cheat Sheet*

NumPy Basics

Learn Python for Data Science Interactively at www.DataCamp.com

NumPy

The **NumPy** library is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays.

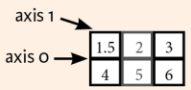
Use the following import convention:

```
>>> import numpy as np
```

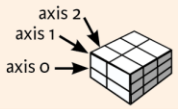
NumPy Arrays

1D array

2D array



3D array



Creating Arrays

```
>>> a = np.array([1,2,3])
>>> b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
>>> c = np.array([[(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]],
                 dtype = float)
```

Initial Placeholders

`>>> np.zeros((3,4))`	Create an array of zeros
`>>> np.ones((2,3,4),dtype=np.int16)`	Create an array of ones
`>>> d = np.arange(10,25,5)`	Create an array of evenly spaced values (step value)
`>>> np.linspace(0,2,9)`	Create an array of evenly spaced values (number of samples)
`>>> e = np.full((2,2),7)`	Create a constant array
`>>> f = np.eye(2)`	Create a 2X2 identity matrix
`>>> np.random.random((2,2))`	Create an array with random values
`>>> np.empty((3,2))`	Create an empty array

I/O

Saving & Loading On Disk

```
>>> np.save('my_array', a)
>>> np.savez('array.npz', a, b)
>>> np.load('my_array.npy')
```

Saving & Loading Text Files

```
>>> np.loadtxt("myfile.txt")
>>> np.genfromtxt("my_file.csv", delimiter=',')
>>> np.savetxt("myarray.txt", a, delimiter=" ")
```

Data Types

`>>> np.int64`	Signed 64-bit integer types
`>>> np.float32`	Standard double-precision floating point
`>>> np.complex`	Complex numbers represented by 128 floats
`>>> np.bool`	Boolean type storing `TRUE` and `FALSE` values
`>>> np.object`	Python object type
`>>> np.string_`	Fixed-length string type
`>>> np.unicode_`	Fixed-length unicode type

Inspecting Your Array

`>>> a.shape`	Array dimensions
`>>> len(a)`	Length of array
`>>> b.ndim`	Number of array dimensions
`>>> e.size`	Number of array elements
`>>> b.dtype`	Data type of array elements
`>>> b.dtype.name`	Name of data type
`>>> b.astype(int)`	Convert an array to a different type

Asking For Help

```
>>> np.info(np.ndarray.dtype)
```

Array Mathematics

Arithmetic Operations

`>>> g = a - b`
`array([[-0.5, 0. , 0. ],`
`[-3. , -3. , -3. ]])`
	Subtraction

`>>> np.subtract(a,b)`	Subtraction

`>>> b + a`
`array([[ 2.5, 4. , 6. ],`
`[ 5. , 7. , 9. ]])`
	Addition

`>>> np.add(b,a)`	Addition

`>>> a / b`
`array([[ 0.66666667, 1. , 1. ],`
`[ 0.25 , 0.4 , 0.5 ]])`
	Division

`>>> np.divide(a,b)`	Division

`>>> a * b`
`array([[ 1.5, 4. , 9. ],`
`[ 4. , 10. , 18. ]])`
	Multiplication

`>>> np.multiply(a,b)`	Multiplication
`>>> np.exp(b)`	Exponentiation
`>>> np.sqrt(b)`	Square root
`>>> np.sin(a)`	Print sines of an array
`>>> np.cos(b)`	Element-wise cosine
`>>> np.log(a)`	Element-wise natural logarithm

`>>> e.dot(f)`
`array([[ 7., 7.],`
`[ 7., 7.]])`
	Dot product

Comparison

`>>> a == b`
`array([[False, True, True],`
`[False, False, False]], dtype=bool)`
	Element-wise comparison

`>>> a < 2` `array([True, False, False], dtype=bool)`	Element-wise comparison
`>>> np.array_equal(a, b)`	Array-wise comparison

Aggregate Functions

`>>> a.sum()`	Array-wise sum
`>>> a.min()`	Array-wise minimum value
`>>> b.max(axis=0)`	Maximum value of an array row
`>>> b.cumsum(axis=1)`	Cumulative sum of the elements
`>>> a.mean()`	Mean
`>>> b.median()`	Median
`>>> a.corrcoef()`	Correlation coefficient
`>>> np.std(b)`	Standard deviation

Copying Arrays

`>>> h = a.view()`	Create a view of the array with the same data
`>>> np.copy(a)`	Create a copy of the array
`>>> h = a.copy()`	Create a deep copy of the array

Sorting Arrays

`>>> a.sort()`	Sort an array
`>>> c.sort(axis=0)`	Sort the elements of an array's axis

Subsetting, Slicing, Indexing

Also see Lists

Subsetting

`>>> a[2]` `3`	1 2 3	Select the element at the 2nd index
`>>> b[1,2]` `6.0`	1.5 2 3 4 5 6	Select the element at row 0 column 2 (equivalent to `b[1][2]`)

Slicing

`>>> a[0:2]` `array([1, 2])`	1 2 3	Select items at index 0 and 1
`>>> b[0:2,1]` `array([ 2., 5.])`	1.5 2 3 4 5 6	Select items at rows 0 and 1 in column 1
`>>> b[:1]` `array([[1.5, 2., 3.]])`	1.5 2 3 4 5 6	Select all items at row 0 (equivalent to `b[0:1, :]`)

`>>> c[1,...]`
`array([[[ 3., 2., 1.],`
`[ 4., 5., 6.]]])`
		Same as `[1,:,:]`

`>>> a[ : :-1]` `array([3, 2, 1])`		Reversed array `a`

Boolean Indexing

`>>> a[a<2]` `array([1])`	1 2 3	Select elements from `a` less than 2

Fancy Indexing

`>>> b[[1, 0, 1, 0],[0, 1, 2, 0]]` `array([ 4. , 2. , 6. , 1.5])`	Select elements `(1,0)`, `(0,1)`, `(1,2)` and `(0,0)`

`>>> b[[1, 0, 1, 0]][:,[0,1,2,0]]`
`array([[ 4. ,5. , 6. , 4. ],`
`[ 1.5, 2. , 3. , 1.5],`
`[ 4. , 5. , 6. , 4. ],`
`[ 1.5, 2. , 3. , 1.5]])`
	Select a subset of the matrix's rows and columns

Array Manipulation

Transposing Array

`>>> i = np.transpose(b)`	Permute array dimensions
`>>> i.T`	Permute array dimensions

Changing Array Shape

`>>> b.ravel()`	Flatten the array
`>>> g.reshape(3,-2)`	Reshape, but don't change data

Adding/Removing Elements

`>>> h.resize((2,6))`	Return a new array with shape (2,6)
`>>> np.append(h,g)`	Append items to an array
`>>> np.insert(a, 1, 5)`	Insert items in an array
`>>> np.delete(a,[1])`	Delete items from an array

Combining Arrays

`>>> np.concatenate((a,d),axis=0)` `array([ 1, 2, 3, 10, 15, 20])`	Concatenate arrays

`>>> np.vstack((a,b))`
`array([[ 1. , 2. , 3. ],`
`[ 1.5, 2. , 3. ],`
`[ 4. , 5. , 6. ]])`
	Stack arrays vertically (row-wise)

`>>> np.r_[e,f]`	Stack arrays vertically (row-wise)

`>>> np.hstack((e,f))`
`array([[ 7., 7., 1., 0.],`
`[ 7., 7., 0., 1.]])`
	Stack arrays horizontally (column-wise)

`>>> np.column_stack((a,d))`
`array([[ 1, 10],`
`[ 2, 15],`
`[ 3, 20]])`
	Create stacked column-wise arrays

`>>> np.c_[a,d]`	Create stacked column-wise arrays

Splitting Arrays

`>>> np.hsplit(a,3)` `[array([1]),array([2]),array([3])]`	Split the array horizontally at the 3rd index

`>>> np.vsplit(c,2)`
`[array([[[ 1.5, 2. , 1. ],`
`[ 4. , 5. , 6. ]]]),`
`array([[[ 3., 2., 3.],`
`[ 4., 5., 6.]]])]`
	Split the array vertically at the 2nd index